

Module 1 | Interpolation



The idea behind interpolation is something you’ve practiced since Kindergarten: Connecting the dots. Now that we’re all grown up, we call our “dots” data points in the xy -plane, and we connect them with very specific kinds of curves.

In the late 1950’s, engineers at two rival French automakers (Paul de Casteljau at Citroen and Pierre Bézier at Renault) used a new way of interpolating data points to generate curves that could be manipulated easily to design automobiles. Their method was a trade secret until Bézier publicized the concept in 1962, which is why we now call the resulting objects “Bézier curves”. The same methods are now commonly used not only in industrial design, but in computer graphics. In particular, most printers use these methods to generate fonts. Here’s a schematic* that illustrates how the outline of an “G” might be drawn by interpolating 23 data points.

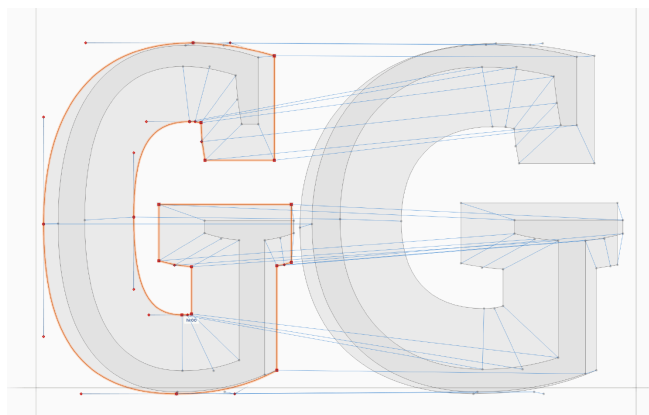


Figure 1.1: Font Curve Mathematics

Notice that the sides are all straight lines and ellipses in this case. Linear functions are first-degree polynomials, and conic sections are second degree. The reason that auto designs and printer fonts look much better than the above is because they use higher-degree polynomials in the interpolation.

§1.1 Polynomial Interpolation

Definition 1.1.1

The set of continuous functions that map from $[a, b] \subseteq \mathbb{R}$ to $\mathbb{R}$ is denoted by $C[a, b]$. The set of continuous functions whose first r derivatives are also continuous on $[a, b]$ is denoted by $C^r[a, b]$. (Note that $C^0[a, b] \equiv C[a, b]$.)

Definition 1.1.2

The set of polynomials of degree n or less is denoted by $\mathcal{P}_n$.

*Taken from <https://www.lucasfonts.com/learn/interpolation-curve-technicalities>

Note that $C[a, b]$, $C^r[a, b]$ (for any $a < b, r \geq 0$) and $\mathcal{P}_n$ are *vector spaces* of functions (since linear combinations of such functions maintain continuity and polynomial degree).

■ Question 1.

□

What is the dimension of the vector space $\mathcal{P}_n$ as a subspace of $C^r[a, b]$?

Working with polynomials is especially convenient on a computer as we only need to store the $n + 1$ coefficients. Operations such as taking the derivative or integrating f are also easier. The idea in this chapter is to then find a polynomial that approximates a general function $f \in C[a, b]$. The following theorem, due to Weierstrass, says that this is always doable!

Theorem 1.1.3: Weierstrass Approximation Theorem

For any $f \in C[a, b]$ and any $\varepsilon > 0$, there exists a polynomial $p_n(x)$ of finite degree such that

$$\max_{x \in [a, b]} |f(x) - p_n(x)| \leq \varepsilon.$$

Sketch of Proof. First rescale f to be defined on $[0, 1]$. Define the n -th Bernstein polynomial as:

$$B_n(f)(x) = \sum_{k=0}^n f\left(\frac{k}{n}\right) B_{k,n}(x)$$

where $B_{k,n}(x) = \binom{n}{k} x^k (1-x)^{n-k}$. Then we can show that for large n , the $|B_n(f)(x) - f(x)|$ is less than ε .

■

In this chapter, we look for a suitable polynomial p_n by using the technique of **interpolation**. The main problem can be stated as:

Given $f \in C[a, b]$ and $n + 1$ distinct points $\{x_j\}_{j=0}^n$ satisfying

$$a \leq x_0 < x_1 < \cdots < x_n \leq b,$$

determine some $p_n \in \mathcal{P}_n$ such that

$$p_n(x_j) = f(x_j) \quad \text{for } j = 0, \dots, n.$$

The points $f(x_j)$ are usually called **nodes** and the polynomial p_n is called a **f -interpolating polynomial**.

As numerical analysts, we seek answers to the following questions:

- Does such a polynomial $p_n \in \mathcal{P}_n$ exist?
- If so, is it unique?
- Does $p_n \in \mathcal{P}_n$ behave like $f \in C[a, b]$ at points $x \in [a, b]$ when $x \neq x_j$ for $j = 0, \dots, n$?
- How can we compute $p_n \in \mathcal{P}_n$ *efficiently* on a computer?

- How can we compute $p_n \in \mathcal{P}_n$ accurately on a computer?
- If we want to add a new interpolation point x_{n+1} , can we easily adjust p_n to give an interpolating polynomial p_{n+1} of one higher degree?
- How should the interpolation points $\{x_j\}$ be chosen?

Regarding this last question, we should note that, in practice, we are not always able to choose the interpolation points as freely as we might like. For example, our 'continuous function $f \in C[a, b]$ ' could actually be a discrete list of previously collected experimental data, and we are stuck with the values $\{x_j\}_{j=0}^n$ at which the data was measured. Least square approximations provide an alternative approach, appropriate for noisy data, where the overall error $|f(x) - p_n(x)|$ is minimised, without requiring p_n to match f at specific points.

1.1.1 Constructing Interpolating Polynomials in Monomial Basis

The most straightforward way of constructing the polynomial interpolant to f is to find $p_n(x) = c_0 + c_1x + c_2x^2 + \dots + c_nx^n$ where c_i comes from solving the linear system

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ 1 & x_2 & x_2^2 & \cdots & x_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} f(x_0) \\ f(x_1) \\ f(x_2) \\ \vdots \\ f(x_n) \end{bmatrix}, \quad (1.1)$$

which we denote as $A\vec{c} = \vec{f}$. Matrices of this form, called Vandermonde matrices, arise in a wide range of applications. Provided all the interpolation points $\{x_j\}$ are distinct, we can show that this matrix is invertible.

■ Question 2.

□

Check that the determinant is given by $\prod_{0 \leq i < j \leq n} (x_i - x_j)$.

One way to solve the above linear system in [equation \(1.1\)](#) is Gaussian elimination. However, this is computationally inefficient (taking $\mathcal{O}(n^3)$ operations). Additionally, the Vandermonde matrix is very ill-conditioned (we will explain later what this means) and as such the results of our numerical computations are remarkably inaccurate.

■ Question 3.

□

Discuss why the coefficients grow in magnitude and oscillate in sign as n grows large.

■ Question 4.

Lab

Write a script that accepts an array of ordered pairs (where each x value is unique) and builds a Vandermonde interpolation polynomial.

- (a) Use your code to build the Vandermonde interpolation polynomial that interpolates the function $f(x) = \cos(2\pi x)$ with $(n + 1)$ points that are linearly spaced on the interval $x \in [0, 2]$. Repeat the experiment with $n = 4, n = 6, n = 8$. Make a plot for each value of n . What do you observe?
- (b) Vandermonde interpolation is relatively easy to conceptualize and code, but there is an inherent problem. Use your Vandermonde interpolation code to create a plot where the horizontal axis is the order of the interpolating polynomial and the vertical axis is the ratio of the maximum eigenvalue to the minimum eigenvalue of the Vandermonde matrix $\left| \frac{\lambda_{\max}}{\lambda_{\min}} \right|$. What does this plot tell you about Vandermonde interpolation for high-order polynomials? You can use the same model function as part (a).

Let's look at two superior bases that give more stable expressions for the interpolant.

1.1.2 Constructing interpolants in the Newton basis

In our first approach, we construct the interpolant by starting with lower degree interpolants for smaller number of data points and then extending it to a new data point.

We start by constructing $p_0 \in \mathcal{P}_0$ such that $p_0(x_0) = c_0$ where $c_0 = f(x_0)$.

Then $p_1 = c_0 + c_1(x - x_1)$ where c_1 is evaluated by using $p_1(x_1) = f(x_1)$.

In general,

$$p_n(x) = p_{n-1}(x) + c_n \prod_{j=0}^{n-1} (x - x_j)$$

and c_n can be evaluated by setting $p_n(x_n) = f(x_n)$.

■ Question 5.

Run the Mathematica script to get a visual sense of the process.

□

■ Question 6.

Prove that the polynomials $\{q_n(x)\}$ where $q_n(x) = \prod_{j=0}^{n-1} (x - x_j)$ are linearly independent. (Hint: use induction.)

□

The polynomials $\{q_n(x)\}$ form a basis of $\mathcal{P}_n$ called the **Newton basis**. So, in this case we are solving a system of the form $A \vec{c} = \vec{f}$ where

$$\begin{bmatrix} 1 & & & & \\ 1 & (x_1 - x_0) & & & \\ 1 & (x_2 - x_0) & (x_2 - x_0)(x_2 - x_1) & & \\ \vdots & \vdots & \vdots & \ddots & \\ 1 & (x_n - x_0) & (x_n - x_0)(x_n - x_1) & \cdots & \prod_{j=0}^{n-1} (x_n - x_j) \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} f(x_0) \\ f(x_1) \\ f(x_2) \\ \vdots \\ f(x_n) \end{bmatrix} \quad (1.2)$$

■ Question 7.

What are the advantages of using the Newton basis over the monomial basis?

□